Torneo de Komikan Concurrente en Tiempo Real

Informe Técnico – Tarea 2 de Sistemas Operativos

Autores: Sebastián Quintero | Alan Troncoso – **Implementación:** C17 con Hilos POSIX y Semáforos de Dijkstra

1. Diseño del Esquema de Sincronización

Comprensión del problema y arquitectura general. El desafío central de esta tarea consiste en llevar el *Komikan* —un juego tradicional mapuche asimétrico donde 1 Puma se enfrenta a 12 Perros— desde un formato por turnos hacia un entorno de competencia en tiempo real. En nuestra solución, programada en C17 dentro de un único proceso utilizando hilos POSIX (`pthread`), no existen turnos: las 13 fichas de cada tablero son hilos independientes que "piensan" durante un retardo aleatorio (entre 50 y 300 ms) e intentan moverse simultáneamente. Al compartir el espacio de memoria del proceso, el problema fundamental es evitar las **condiciones de carrera** (*race conditions*), es decir, que dos fichas lean y modifiquen una misma casilla al mismo tiempo dejando el tablero en un estado inconsistente.

Al diseñar la arquitectura, aplicamos en la práctica la diferencia conceptual entre las dos primitivas clásicas de sincronización vistas en el curso: utilizamos **semáforos contadores de Dijkstra** (`sem_t` sin nombre, inicializados con `pshared=0`) cuando necesitamos coordinar la espera pasiva por una *cantidad* de recursos o eventos disponibles, y utilizamos **mutex** (`pthread_mutex_t`) cuando requerimos **exclusión mutua** estricta para que solo un hilo a la vez pueda leer o modificar una estructura de datos compartida.

Mecanismo	Tipo y valor inicial	Función dentro del Torneo y recurso que protege
`sem_cola`	Semáforo Dijkstra (0)	Cuenta cuántos clientes han llegado a la fila y están listos para ser emparejados por el *matchmaker*.
`mutex_cola`	Mutex global	Protege el arreglo compartido `cola[]` y el índice `cola_fin` para que dos clientes no sobrescriban su llegada.
`sem_mesas`	Semáforo Dijkstra (M)	Representa los cupos de mesas libres. Limita el servidor a un máximo de M partidas activas en paralelo.
`mutex_mesas`	Mutex global	Protege la bandera `ocupada` de cada mesa cuando el *matchmaker* busca una libre o el Juez la desocupa.
`Mesa.mutex`	Mutex local (1 por mesa)	Garantiza exclusión mutua sobre la matriz `tablero[7][5]`, posiciones de las 13 fichas, capturas y banderas de fin.
`mutex_log`	Mutex global	Serializa la escritura en consola y en `torneo.log` para evitar que los mensajes de distintas mesas se mezclen.
`terminar`	`volatile sig_atomic_t`	Bandera global ($0 \to 1$) activada por el manejador de `SIGINT` para ordenar el cierre limpio del servidor.

Emparejamiento (Productor–Consumidor) y control de mesas. Para gestionar la llegada de los N jugadores sin desperdiciar ciclos de CPU preguntando continuamente si ya llegaron (espera activa o *busy waiting*), implementamos el patrón **productor–consumidor**. Cuando un `hilo_cliente` llega al servidor tras su retardo inicial, adquiere `mutex_cola`, escribe su identificador en `cola[cola_fin++]` y libera el mutex; acto seguido ejecuta `sem_post(&sem_cola)` (la operación V o *signal* de Dijkstra), incrementando en 1 el contador de jugadores en espera. Por su parte, el `hilo_matchmaker` actúa como consumidor ejecutando dos veces la operación P o *wait* sobre `sem_cola`: si aún no hay dos jugadores en la fila, el sistema operativo bloquea al hilo de forma pasiva hasta que se complete una pareja (el primero en llegar juega como Puma y el segundo controla a los Perros; si N es impar, el jugador sobrante se reporta al final).

Una vez reunida la pareja, el *matchmaker* solicita una mesa ejecutando un *wait* sobre `sem_mesas`, cuyo contador parte en M. Si las M mesas están ocupadas, el contador vale 0 y la pareja espera pasivamente sin consumir CPU. Cuando el semáforo concede el paso, sabemos con certeza que existe al menos una mesa libre; el *matchmaker* toma brevemente `mutex_mesas` para buscar el índice con `ocupada == 0`, lo marca en 1 y crea un `hilo_juez` para dirigir esa partida. Al terminar el encuentro, el Juez vuelve a poner `ocupada = 0` bajo `mutex_mesas` y ejecuta `sem_post(&sem_mesas)`, liberando el cupo para la siguiente pareja.

Exclusión mutua en el tablero y ciclo del Juez. El tablero de 28 intersecciones válidas (5×5 más el triángulo del trono) se representa en una matriz de 7×5 donde las coordenadas fuera de la figura se marcan como `INVALIDA`. Para impedir que dos piezas reclamen la misma casilla a la vez, cada mesa cuenta con su propio `Mesa.mutex`. En este punto aplicamos una regla esencial de diseño concurrente: **el retardo aleatorio de reflexión (50–300 ms) se ejecuta siempre fuera de la sección crítica**. Cuando una ficha despierta de su `nanosleep`, toma `Mesa.mutex`, comprueba que `activa == 1` (y que no haya sido capturada, en el caso de los perros), calcula sus destinos libres y efectúa el movimiento dentro del mismo bloqueo antes de hacer `unlock`.

Encerrar la verificación y el movimiento dentro del mismo `lock` elimina la vulnerabilidad clásica *check-then-act*: si dos Perros desean moverse a la misma intersección vacía, el primero que adquiere `Mesa.mutex` actualiza la matriz a `PERRO`; cuando el segundo hilo entra a su sección crítica un instante después y llama a `destinos_libres()`, ya observa la casilla

ocupada y escoge otra opción válida. Del mismo modo, el salto de captura del Puma (vaciar la casilla de origen, retirar al perro saltado marcando `perro_vivo[p] = 0`, ocupar la casilla de destino e incrementar `capturados++`) ocurre como una única operación atómica frente al resto de los hilos, evitando que se observe un estado intermedio inválido.

Por su parte, el `hilo_juez` inicializa el mutex y el tablero (`preparar_mesa`) antes de lanzar los 13 hilos de las fichas. Durante la partida, el Juez duerme de a 40 ms y, al despertar, toma `Mesa.mutex` para evaluar las condiciones de término: abandono por fallo de algún jugador (`fallo_puma` o `fallo_perros`), victoria del Puma (`capturados >= 6`), victoria de los Perros (Puma acorralado sin movimientos posibles según `puma_tiene_movimientos()`) o empate al alcanzar `max_movs`. Cuando se cumple alguna condición —o si el usuario presiona Ctrl+C (`SIGINT`)—, el Juez cambia `activa = 0`, suelta el mutex, espera con `pthread_join` a que las 13 fichas finalicen limpiamente, destruye `Mesa.mutex` y libera la mesa.

2. Análisis de Riesgos de Concurrencia

- **Prevención de Interbloqueo (*Deadlock*):** Para que ocurra un *deadlock* deben cumplirse simultáneamente las cuatro condiciones de Coffman. Dado que la exclusión mutua y la no apropiación son propias de los mutex POSIX, diseñamos el código para romper las dos condiciones restantes:
  (a) *Ruptura de Retención y Espera:* Ningún hilo se bloquea esperando un semáforo mientras retiene un mutex. El *matchmaker* espera en `sem_cola` y `sem_mesas` sin tener ningún candado tomado, y el Juez libera explícitamente `Mesa.mutex` *antes* de invocar `pthread_join` sobre los hilos de las fichas (si conservara el mutex tomado durante el `join`, cualquier ficha que despertara e intentara hacer `lock` para salir quedaría bloqueada esperando al Juez, mientras el Juez espera a la ficha, causando un *deadlock* clásico). Asimismo, si una ficha simula un fallo interno (`ficha_falla`), levanta la bandera de abandono y hace `pthread_mutex_unlock(&m->mutex)` antes de terminar su hilo.
  (b) *Ruptura de Espera Circular:* En casi todo el programa se adquiere un único mutex a la vez. El único lugar donde se anidan dos candados es al imprimir eventos de una partida: la ficha o el Juez tiene tomado `Mesa.mutex` y llama a `log_msg()`, que internamente adquiere `mutex_log`. Como `mutex_log` actúa como un candado "hoja" (mientras está tomado jamás solicita ningún otro mutex) y ninguna parte del código adquiere `Mesa.mutex` teniendo `mutex_log`, existe un orden total estricto (`Mesa.mutex` → `mutex_log`) que hace imposible la formación de ciclos.
- **Prevención de Bloqueo Activo (*Livelock*):** Un *livelock* se produciría si las fichas reintentaran moverse en un ciclo continuo sin dormir cuando no encuentran casillas libres, o si dos piezas se cedieran el paso mutuamente sin avanzar. En nuestra implementación, si una ficha toma el mutex y no tiene destinos válidos, simplemente libera el candado y vuelve a dormir un tiempo aleatorio usando su propia semilla independiente (`rand_r(&seed)` inicializada con `seed + partida*100 + idx`), lo que desincroniza a los hilos entre sí. A nivel de la dinámica del Komikan, advertimos que como los Perros eligen movimientos al azar, una partida podría prolongarse indefinidamente repitiendo posiciones; por ello añadimos el límite `MAX_MOVS` (250 por defecto), que cierra el juego en empate y garantiza el progreso del torneo. Finalmente, para que la señal `SIGINT` no deje al *matchmaker* bloqueado indefinidamente en `sem_wait`, encapsulamos la espera en `esperar_sem()` mediante `sem_timedwait` con plazos de 100 ms, permitiendo verificar la bandera `terminar` y abortar limpiamente.
- **Mitigación de Inanición (*Starvation*):** Aunque los mutex POSIX estándar no garantizan una cola estricta de tipo FIFO para los hilos que compiten por un candado, mitigamos el riesgo de inanición haciendo que las secciones críticas sean sumamente breves (solo operaciones en memoria y registro de texto) frente al tiempo que cada ficha permanece dormida fuera del mutex (entre 50 y 300 ms, con una media de 175 ms). Al estar el candado libre más del 90 % del tiempo, la probabilidad de que una misma ficha pierda repetidamente el acceso al tablero es despreciable. En la fila de entrada tampoco existe inanición, ya que `cola[]` se procesa en estricto orden de llegada.

3. Evaluación del Diseño

Balance entre complejidad e independencia de bloqueos (Granularidad). Durante el desarrollo evaluamos la alternativa de colocar un mutex por cada una de las 28 casillas del tablero en vez de un único `Mesa.mutex` por partida. Descartamos la granularidad por casilla porque incrementaba enormemente la complejidad y el riesgo de *deadlock*: un salto de captura del Puma involucra tres casillas simultáneas (origen, perro intermedio y destino), lo que habría exigido bloquear tres mutex en un orden canónico de coordenadas, además de requerir candados adicionales para proteger los contadores globales y verificar si el Puma quedó acorralado. Dado que en una mesa solo compiten 13 fichas que pasan la mayor parte del tiempo durmiendo, usar un único mutex por mesa mantiene el código simple, seguro y verificable, mientras que el paralelismo real del servidor se aprovecha **entre las M mesas activas**, donde cada partida opera con su propio `Mesa.mutex` sin bloquear a las demás.

Rendimiento (*Throughput* y Latencia) y decisiones de arbitraje. La latencia de cada movimiento está dominada por el retardo aleatorio configurado (50–300 ms), por lo que la espera para adquirir el mutex de la mesa es imperceptible.

El único punto de serialización global entre mesas distintas es `mutex_log`, el cual se invoca dentro de la sección crítica del tablero. Tomamos esta decisión de forma consciente: aunque realizar la escritura en consola y en `torneo.log` con `Mesa.mutex` tomado extiende ligeramente la duración de la sección crítica por el costo de entrada/salida, esto nos garantiza que las impresiones del tablero reflejen una instantánea 100 % coherente y que el archivo de registro conserve el orden cronológico exacto en que ocurrieron las jugadas.

Por último, elegir un esquema de monitoreo periódico en el Juez (revisando el estado cada 40 ms) en lugar de despertar al árbitro mediante variables de condición (`pthread_cond_t`) tras cada jugada nos permitió desacoplar por completo la lógica de movimiento de las fichas de las reglas de arbitraje, asumiendo un desfase máximo de apenas 40 ms al detectar la victoria. Validamos nuestra implementación compilando sin advertencias (`-Wall -Wextra -std=c17`) y ejecutando pruebas intensivas con *ThreadSanitizer* y *AddressSanitizer/LeakSanitizer* (incluyendo abandonos con `PROB_FALLO` e interrupciones con `SIGINT`), comprobando la ausencia total de condiciones de carrera, recursos huérfanos y fugas de memoria.